How Does Git Work, Internally?

Git is a popular version-controlled tool. However, Git commands have always been confusing, and most users stumble over the terminologies. After diving deep into its internals in this tutorial, you will be able to use commands like git commit, git cherry-pick, etc, with a new perspective.

Git can maintain multiple versions of a file since it stores them all in its database. It creates a new database for each Git repository, which is present in the *.git* folder.

Let's create a new repository as *git_tutorial_dir*. A repository isn't a Git repository until we initialise it. To initialise a repository, the *git init* command is used. This command creates a Git database in the repository.

```
→ Downloads mkdir git_tutorial_dir
→ Downloads cd git_tutorial_dir
→ git_tutorial_dir ls
→ git_tutorial_dir ls -a
.   ..
→ git_tutorial_dir git init
Initialized empty Git repository in
/Downloads/git_tutorial_dir/.git/
→ git_tutorial_dir git:(main) ls -a
.  .. .git
→ git_tutorial_dir git:(main)
```

To understand Git, let's understand Git objects first.

Git objects

Every file or directory is stored as an object in Git. It stores these Git objects as a key value pair, where the key is the object ID and value is the object. Let's understand each of these objects in detail.

Blobs: For each file in a repository, Git creates a blob object. Blob objects contain each line present in a file. The only difference between a blob object and a file is that unlike the latter, a blob does not store, create and update the time stamp.

The object ID of a blob object is a SHA-1 hash value. This hash value is unique among all the objects created in the repository. It is a checksum of the content stored in the file plus a header. The first two characters of the object ID are used as a directory and the remaining 38 characters are used as a file name. There can be at the most 256 directories present in the *.git/objects* folder. This is done to quickly do a binary search for any object whenever required.

```
git_tutorial_dir git:(main) ls
git_tutorial_dir git:(main) touch file1.txt
git_tutorial_dir git:(main) x echo "Adding first line to file1"
> file1.txt
git_tutorial_dir git:(main) x cd .git/objects
objects git: (main) tree
|
|---info
|__pack
3 directories, 0 files
objects git:(main)
```

Here I have created a new file as *file1.txt* and added some content to it. But when I checked the Git objects folder nothing was present in it.

```
git_tutorial_dir git:(main) × git add file1.txt
git_tutorial_dir git:(main) x
git_tutorial_dir git:(main) x cd .git/objects
objects git: (main) tree
.
|_ec
   |__ 7ff284664263109178b5ea42eadfef0fa8e8e2
|__info
```